OpenMP runtime library

OpenMP runtime library

- OpenMP provides compiler directives, environment variables and a runtime library with functions we can call directly from our programs
- The previous unit demonstrated the use of a compiler directive and an environment variable
- In this unit we will see an example of how to use the OpenMP runtime library

What do the OpenMP runtime library routines do?

- The OpenMP 4.5 specification includes:
  - Routines to control and query the execution environment (36 routines)
  - Timing routines (2 routines)
  - Device memory routines for managing memory and pointers on accelerators e.g. GPUs (7 routines)
  - Locking routines to synchronise access to data (12 routines)

Querying the execution environment

- The following example uses three functions for querying the execution environment
- To make these functions available to our C program we will include the `omp.h` header file: `#include <omp.h>`

`int omp_get_thread_num(void)`	Returns the thread number of the calling thread
`int omp_get_num_threads(void)`	Returns the total number of threads
`int omp_get_num_procs(void)`	Returns the number of processors available

Runtime library example

```
#include <stdio.h>
#include <omp.h>

int main (){

  printf( "Outside: there are %d processors available\n", omp_get_num_procs() );
  printf( "Outside: I am thread %d of %d\n", omp_get_thread_num(), omp_get_num_threads() );

#pragma omp parallel
  {
  printf( "Inside: there are %d processors available\n", omp_get_num_procs() );
  printf( "Inside: I am thread %d of %d\n", omp_get_thread_num(), omp_get_num_threads() );
  }

  return 0;
}
```

Demo

```
[ml795@blue24 ~]$ cd HPC
[ml795@blue24 HPC]$ ls
hello  hello.c  rtl.c
[ml795@blue24 HPC]$ cat rtl.c
/*****************************************************************
Description: The run time library example program from
             ECM3446/ECMM461 unit 2.3

Purpose: demonstrate the use of the OpenMP run time library

Notes: compile with the appropriate OpenMP flag for
       your compiler e.g. gcc -fopenmp -o hello hello.c


*****************************************************************/

#include <stdio.h>
#include <omp.h>

int main (){

  printf( "Outside: there are %d processors available\n", omp_get_num_procs() );
  printf( "Outside: I am thread %d of %d\n", omp_get_thread_num(), omp_get_num_threads() );

#pragma omp parallel
  {
  printf( "Inside: there are %d processors available\n", omp_get_num_procs() );
  printf( "Inside: I am thread %d of %d\n", omp_get_thread_num(), omp_get_num_threads() );
  }

  return 0;
}


[ml795@blue24 HPC]$
```

Demo

```
[ml795@blue24 ~]$ cd HPC
[ml795@blue24 HPC]$ ls
hello  hello.c  rtl.c
[ml795@blue24 HPC]$ cat rtl.c
/*****************************************************************
Description: The run time library example program from
             ECM3446/ECMM461 unit 2.3

Purpose: demonstrate the use of the OpenMP run time library

Notes: compile with the appropriate OpenMP flag for
       your compiler e.g. gcc -fopenmp -o hello hello.c


*****************************************************************/

#include <stdio.h>
#include <omp.h>

int main (){

  printf( "Outside: there are %d processors available\n", omp_get_num_procs() );
  printf( "Outside: I am thread %d of %d\n", omp_get_thread_num(), omp_get_num_threads() );

#pragma omp parallel
  {
  printf( "Inside: there are %d processors available\n", omp_get_num_procs() );
  printf( "Inside: I am thread %d of %d\n", omp_get_thread_num(), omp_get_num_threads() );
  }

  return 0;
}


[ml795@blue24 HPC]$ gcc -fopenmp -o rtl rtl.c
[ml795@blue24 HPC]$ ls
hello  hello.c  rtl  rtl.c
[ml795@blue24 HPC]$
```

Demo

```
  printf( "Outside: there are %d processors available\n", omp_get_num_procs() );
  printf( "Outside: I am thread %d of %d\n", omp_get_thread_num(), omp_get_num_threads() );

#pragma omp parallel
  {
  printf( "Inside: there are %d processors available\n", omp_get_num_procs() );
  printf( "Inside: I am thread %d of %d\n", omp_get_thread_num(), omp_get_num_threads() );
  }

  return 0;
}


[[ml795@blue24 HPC]$ gcc -fopenmp -o rtl rtl.c                                                ]
[[ml795@blue24 HPC]$ ls                                                                       ]
hello  hello.c  rtl  rtl.c
[[ml795@blue24 HPC]$ ./rtl                                                                    ]
Outside: there are 16 processors available
Outside: I am thread 0 of 1
Inside: there are 16 processors available
Inside: there are 16 processors available
Inside: I am thread 4 of 16
Inside: there are 16 processors available
Inside: I am thread 2 of 16
Inside: there are 16 processors available
Inside: I am thread 1 of 16
Inside: there are 16 processors available
Inside: I am thread 3 of 16
Inside: there are 16 processors available
Inside: I am thread 5 of 16
Inside: there are 16 processors available
Inside: I am thread 15 of 16
Inside: there are 16 processors available
Inside: I am thread 14 of 16
Inside: there are 16 processors available
Inside: I am thread 12 of 16
Inside: there are 16 processors available
Inside: I am thread 10 of 16
Inside: I am thread 0 of 16
Inside: there are 16 processors available
Inside: I am thread 9 of 16
Inside: there are 16 processors available
Inside: I am thread 8 of 16
Inside: there are 16 processors available
Inside: I am thread 6 of 16
Inside: there are 16 processors available
Inside: I am thread 13 of 16
Inside: there are 16 processors available
Inside: I am thread 11 of 16
Inside: there are 16 processors available
Inside: I am thread 7 of 16
[ml795@blue24 HPC]$
```

Demo

```
Inside: there are 16 processors available
Inside: I am thread 12 of 16
Inside: there are 16 processors available
Inside: I am thread 10 of 16
Inside: I am thread 0 of 16
Inside: there are 16 processors available
Inside: I am thread 9 of 16
Inside: there are 16 processors available
Inside: I am thread 8 of 16
Inside: there are 16 processors available
Inside: I am thread 6 of 16
Inside: there are 16 processors available
Inside: I am thread 13 of 16
Inside: there are 16 processors available
Inside: I am thread 11 of 16
Inside: there are 16 processors available
Inside: I am thread 7 of 16
[ml795@blue24 HPC]$ ./rtl
Outside: there are 16 processors available
Outside: I am thread 0 of 1
Inside: there are 16 processors available
Inside: there are 16 processors available
Inside: I am thread 8 of 16
Inside: there are 16 processors available
Inside: I am thread 10 of 16
Inside: there are 16 processors available
Inside: I am thread 14 of 16
Inside: there are 16 processors available
Inside: I am thread 3 of 16
Inside: I am thread 0 of 16
Inside: there are 16 processors available
Inside: I am thread 7 of 16
Inside: there are 16 processors available
Inside: I am thread 1 of 16
Inside: there are 16 processors available
Inside: I am thread 15 of 16
Inside: there are 16 processors available
Inside: I am thread 2 of 16
Inside: there are 16 processors available
Inside: I am thread 4 of 16
Inside: there are 16 processors available
Inside: I am thread 11 of 16
Inside: there are 16 processors available
Inside: I am thread 6 of 16
Inside: there are 16 processors available
Inside: I am thread 5 of 16
Inside: there are 16 processors available
Inside: I am thread 13 of 16
Inside: there are 16 processors available
Inside: I am thread 12 of 16
Inside: there are 16 processors available
Inside: I am thread 9 of 16
[ml795@blue24 HPC]$
```

Unit Summary

- The OpenMP runtime library provides functions which can be called from C program
- To make these functions available include the `omp.h` header file
- Ran an example program which used runtime library routines to query the execution environment
- The order in which threads executed the print statements in the parallel region varied